PÓS TECH

ARQUITETURA E DESENVOLVIMENTO JAVA

TECH CHALLENGE — FASE 2

Sistema de Gestão para Restaurantes — Tipos de usuário, restaurantes e cardápio

Equipe: Restaurant Management — FIAP Postech

Autores:

- José Bernardino da Silva Júnior
- Marcos Ruiz Freire
- Lucas Brisolla
- Rodrigo Fittipaldi

Data: 02/11/2026

Repositório: https://github.com/rodrfit/restaurant-management

1. Introdução

Este relatório descreve a solução desenvolvida para a Fase 2 do Tech Challenge da Pós Tech em Arquitetura e Desenvolvimento Java (FIAP). O desafio propõe a construção de um sistema de gestão compartilhado para um grupo de restaurantes da região, motivado pelo alto custo de sistemas individuais. Os estabelecimentos usam uma plataforma única, e os clientes escolhem o restaurante pela comida oferecida.

Por limitação de recursos, a entrega foi acordada em fases. A Fase 1 entregou o módulo de usuários: cadastro, atualização, exclusão, busca por nome, última alteração, e-mail único, validação de login e troca de senha em endpoint separado. Nesta Fase 2 o escopo acrescenta a gestão de tipos de usuário, a associação do tipo a um usuário existente, o cadastro de restaurantes e o cadastro dos itens de cardápio.

A aplicação permanece em Java 21 com Spring Boot 4.1, MySQL 8, Docker Compose, Swagger/OpenAPI, coleção Postman e erros no padrão ProblemDetail (RFC 7807). A API continua versionada em /api/v1. O código foi reorganizado em Clean Architecture (domain, application e infrastructure).

2. Definição do Problema

Na região, um grupo de restaurantes decidiu contratar estudantes para construir um sistema de gestão único, em vez de pagar um sistema separado para cada casa. O cliente deve escolher o restaurante pela comida, e não pela qualidade do software de cada estabelecimento.

A entrega em fases permite implementar aos poucos e ajustar o que os restaurantes e os clientes usarem. Nesta fase o objetivo é expandir o backend com três cadastros e manter a organização do código, a documentação, os testes e a execução via Docker.

2.1 Objetivo

• Cadastrar tipos de usuário, com o nome do tipo, e associar um tipo a um usuário já existente. Os perfis previstos são Cliente e Dono de Restaurante.

• Cadastrar restaurantes com nome, endereço, tipo de cozinha, horário de funcionamento e dono (um usuário existente).

• Cadastrar itens de cardápio com nome, descrição, preço, indicação de consumo apenas no restaurante e o caminho da foto do prato. O arquivo da imagem não é armazenado.

• Manter o que a Fase 1 já entregou: usuários, senha, login, e-mail único e ProblemDetail.

• Organizar o código em camadas (domain, application, infrastructure).

• Executar aplicação e MySQL com Docker Compose.

3. Arquitetura da Aplicação

3.1 Visão geral

A aplicação é um serviço REST em Spring Boot 4.1 e Java 21. A dependência aponta para dentro:

```
infrastructure -> application -> domain
```

O caminho de uma requisição é:

```
HTTP -> Controller -> Service -> porta de repositório -> adapter JPA -> MySQL
```

• domain: modelos (User, UserType, Restaurant, MenuItem, Address), portas de repositório, porta PasswordHasher e exceções de negócio. Não usa Spring nem JPA.

• application: casos de uso. Os serviços recebem commands e devolvem objetos de domínio.

• infrastructure: controllers e DTOs (web), entidades JPA, repositórios Spring Data, mappers e adapters (persistence), BcryptPasswordHasher (security) e OpenAPI (config).

O ArchitectureTest (ArchUnit) impede que domain dependa de Spring, JPA, application ou infrastructure, e impede que application dependa de infrastructure.

Erros saem pelo GlobalExceptionHandler (@RestControllerAdvice) no formato ProblemDetail (RFC 7807). Senhas são hash BCrypt. A troca de senha continua em endpoint separado da atualização cadastral.

3.2 Estrutura de pacotes

```
src/main/java/com/restaurant/management/
├── domain/
│   ├── model/
│   ├── repository/
│   ├── port/
│   └── exception/
├── application/
│   ├── command/
│   └── service/
└── infrastructure/
    ├── web/
    ├── persistence/
    ├── security/
```

```
└── config/
```

4. Modelagem

O tipo do usuário é a entidade UserType, ligada ao usuário pela coluna user_type_id. Não há mais a herança Client / RestaurantOwner. O endereço do usuário é um valor embutido.

4.1 Tipo de usuário (user_types)

Atributo	Tipo	Observação
id	Long	Chave gerada
name	String	Obrigatório e único. É o campo pedido no enunciado
observation	String	Opcional, até 1000 caracteres

4.2 Usuário (users)

Atributo	Tipo	Observação
id	Long	Chave gerada
name	String	Obrigatório
email	String	Obrigatório e único
login	String	Obrigatório e único
password	String	Hash BCrypt, 60 caracteres
last_change	LocalDateTime	Atualizado a cada gravação
user_type_id	Long	Chave estrangeira para user_types
street, number, city, zip_code	String	Endereço embutido

4.3 Restaurante (restaurants)

Atributo	Tipo	Observação
id	Long	Chave gerada
nome	String	Obrigatório
endereco	String	Obrigatório
tipo_cozinha	String	Obrigatório
horario_funcionamento	String	Obrigatório
owner_id	Long	Usuário responsável pelo restaurante

4.4 Item de cardápio (menu_items)

Atributo	Tipo	Observação
id	Long	Chave gerada
nome	String	Obrigatório
descricao	String	Obrigatório
preco	Double	Maior que zero
disponivel_no_local	Boolean	Consumo apenas no restaurante
foto_path	String	Caminho da foto. O arquivo não é salvo
restaurant_id	Long	Restaurante do item

As tabelas são criadas pelo Hibernate com ddl-auto: update.

Bancos criados na Fase 1 ainda possuem a coluna users_type (NOT NULL), usada pela herança antiga. O update do Hibernate não remove coluna. Nesses bancos o cadastro de

usuário responde 409. O volume local é recriado com docker compose down -v.

5. Endpoints

A base é http://localhost:8080. A documentação interativa fica em http://localhost:8080/swagger-ui.html.

Método	Endpoint	Descrição
POST	/api/v1/auth/login	Valida login e senha
POST	/api/v1/user-types	Cria tipo de usuário
GET	/api/v1/user-types	Lista tipos
GET	/api/v1/user-types/{id}	Busca tipo por id
PUT	/api/v1/user-types/{id}	Atualiza tipo
DELETE	/api/v1/user-types/{id}	Remove tipo
POST	/api/v1/users	Cadastra usuário já associado a um tipo
GET	/api/v1/users	Lista usuários. Filtro opcional ?name=
GET	/api/v1/users/{id}	Busca usuário por id
PUT	/api/v1/users/{id}	Atualiza nome, e-mail e endereço
PUT	/api/v1/users/{id}/password	Troca a senha
PUT	/api/v1/users/{id}/type/{typeId}	Associa um tipo existente ao usuário
DELETE	/api/v1/users/{id}	Remove usuário
POST	/api/v1/restaurants	Cria restaurante com dono
GET	/api/v1/restaurants	Lista restaurantes
GET	/api/v1/restaurants/{id}	Busca restaurante por id
PUT	/api/v1/restaurants/{id}	Atualiza restaurante
DELETE	/api/v1/restaurants/{id}	Remove restaurante
POST	/api/v1/menu-items	Cria item de cardápio
GET	/api/v1/menu-items	Lista itens
GET	/api/v1/menu-items/{id}	Busca item por id
PUT	/api/v1/menu-items/{id}	Atualiza item
DELETE	/api/v1/menu-items/{id}	Remove item

5.1 Tipo de usuário (POST /api/v1/user-types)

Requisição:

```
{
  "name": "Cliente",
  "observation": "Usuário que consulta o cardápio"
}
```

Resposta 201 Created:

```
{
  "id_tipo_usuario": 1,
  "name": "Cliente",
  "observation": "Usuário que consulta o cardápio"
}
```

Nome duplicado responde 409. Nome ausente responde 400.

5.2 Cadastro de usuário (POST /api/v1/users)

O campo id_tipo_usuario aponta para um tipo já cadastrado.

```
{
  "name": "João Silva",
  "email": "joao.silva@email.com",
  "login": "joaosilva",
  "password": "senha123",
  "id_tipo_usuario": 1,
  "address": {
    "street": "Rua das Flores",
    "number": "123",
    "city": "São Paulo",
    "zipCode": "01310-100"
  }
}
```

Resposta 201 Created (campos principais):

```
{
  "id_usuario": 1,
  "name": "João Silva",
  "email": "joao.silva@email.com",
  "login": "joaosilva",
  "tipo_usuario": "Cliente",
  "id_tipo_usuario": 1,
  "descricao_tipo_usuario": "Usuário que consulta o cardápio",
  "address": {
    "street": "Rua das Flores",
    "number": "123",
    "city": "São Paulo",
    "zipCode": "01310-100"
  }
}
```

E-mail duplicado responde 409 com ProblemDetail. Tipo inexistente responde 404.

5.3 Associação de tipo (PUT /api/v1/users/{id}/type/{typeId})

Associa um tipo já cadastrado a um usuário existente. Resposta 200 com o usuário atualizado. Usuário ou tipo inexistente responde 404.

5.4 Atualização cadastral e senha

PUT /api/v1/users/{id} aceita apenas name, email e address.

PUT /api/v1/users/{id}/password:

```
{
  "actualPassword": "senha123",
  "newPassword": "senhaNova456"
}
```

Resposta 204 sem corpo. Senha atual inválida responde 400. Usuário inexistente responde 404.

5.5 Login (POST /api/v1/auth/login)

```
{
  "login": "joaosilva",
  "password": "senha123"
}
```

Resposta 200:

```
{
  "valid": true,
```

```
  "message": "Login válido",
  "userId": 1,
  "type": "Cliente"
}
```

Credencial inválida também responde 200, com valid: false e a mensagem "Login ou senha inválidos". Não há sessão.

5.6 Restaurante (POST /api/v1/restaurants)

```
{
  "nome": "Sabor do Bairro",
  "endereco": "Rua das Flores, 123",
  "tipoCozinha": "Italiana",
  "horarioFuncionamento": "Segunda a domingo, 18:00-23:00",
  "ownerId": 1
}
```

Resposta 201 devolve os mesmos campos mais id. Dono inexistente responde 404. Campo obrigatório ausente responde 400.

5.7 Item de cardápio (POST /api/v1/menu-items)

```
{
  "nome": "Pizza Margherita",
  "descricao": "Pizza artesanal com molho de tomate e mussarela",
  "preco": 39.90,
  "disponivelNoLocal": true,
  "fotoPath": "/images/pizza-margherita.jpg",
  "restaurantId": 1
}
```

fotoPath é apenas o caminho. disponivelNoLocal indica consumo só no restaurante. Restaurante inexistente responde 404.

5.8 Erro padronizado

```
{
  "type": "about:blank",
  "title": "Conflict",
  "status": 409,
  "detail": "E-mail já cadastrado: joao.silva@email.com"
}
```

6. Documentação Swagger

A especificação é gerada com springdoc-openapi. Com a aplicação na porta 8080:

- Swagger UI: http://localhost:8080/swagger-ui.html
- OpenAPI JSON: http://localhost:8080/v3/api-docs
- Base da API: http://localhost:8080/api/v1

As tags são Autenticação, Usuários, Tipos de Usuário, Restaurantes e Itens do Menu. Os endpoints declaram respostas de sucesso e de erro (400, 404 e 409) com ProblemDetail.

7. Coleção Postman

A coleção da Fase 2 está em docs/postman/RestaurantManagement.fase2.2.postman_collection.json. A variável baseUrl vale http://localhost:8080.

Pastas e requisições:

- Authentication: Login
- Users: Create User, List Users, Get User By Id, Update User, Update User Password, Delete User
- User Types: Create User Type, List User Types, Get User Type By Id, Update User Type, Delete User Type, Associate User Type
- Restaurants: Create Restaurant, List Restaurants, Get Restaurant By Id, Update Restaurant, Delete Restaurant
- Menu Items: Create Menu Item, List Menu Items, Get Menu Item By Id, Update Menu Item, Delete Menu Item

A ordem útil de teste é: criar o tipo, criar o usuário com id_tipo_usuario, associar o tipo se necessário, criar o restaurante com ownerId e criar o item com restaurantId.

8. Banco de dados

MySQL 8. O Hibernate atualiza o esquema com ddl-auto: update.

Tabela	Conteúdo
user_types	id, name (único), observation
users	id, name, email (único), login (único), password (hash), last_change, user_type_id, street, number, city, zip_code
restaurants	id, nome, endereco, tipo_cozinha, horario_funcionamento, owner_id
menu_items	id, nome, descricao, preco, disponivel_no_local, foto_path, restaurant_id

E-mail, login e nome do tipo são únicos também no banco. A senha não é gravada em texto puro.

9. Execução com Docker Compose

A aplicação publica a porta 8080. O MySQL do container é publicado na porta 3307 do host (3307:3306).

9.1 Endereços

- Aplicação: http://localhost:8080
- Swagger UI: http://localhost:8080/swagger-ui.html
- OpenAPI: http://localhost:8080/v3/api-docs
- API: http://localhost:8080/api/v1
- MySQL no host: localhost:3307, banco restaurant_management, usuário management

Exemplo de .env na raiz (o arquivo versionado é .env.example):

```
MYSQL_ROOT_PASSWORD=root
MYSQL_DATABASE=restaurant_management
MYSQL_USER=management
MYSQL_PASSWORD=management123
```

O healthcheck do Compose usa o usuário root e a senha root. MYSQL_ROOT_PASSWORD

precisa ser root.

9.2 Passo a passo

1. Clonar https://github.com/rodrfit/restaurant-management.git e entrar na pasta.
2. Copiar o exemplo: cp .env.example .env.
3. Subir aplicação e banco: docker compose up --build -d.
4. Aguardar o MySQL ficar saudável e a aplicação responder em http://localhost:8080.
5. Abrir o Swagger em http://localhost:8080/swagger-ui.html.
6. Importar docs/postman/RestaurantManagement.fase2.2.postman_collection.json e executar os fluxos.

Para rodar só o banco no Docker e a aplicação na máquina:

```
docker compose up -d mysql
./mvnw spring-boot:run
```

A aplicação local conecta em jdbc:mysql://localhost:3307/restaurant_management com usuário management e senha management123.

docker compose down encerra os containers. docker compose down -v apaga também o volume do MySQL.

10. Testes

Os testes ficam em src/test/java e usam JUnit 5 e Mockito. O teste de contexto sobe o Spring com H2 em memória e não depende do MySQL. Execução: ./mvnw test.

Classes:

- UserServiceImplTest — cadastro, busca, atualização, senha, exclusão e associação de tipo
- UserTypeServiceImplTest — criação e atualização de tipo
- RestaurantServiceImplTest — criação de restaurante com dono
- MenuItemServiceImplTest — criação de item ligado a um restaurante
- AuthServiceImplTest — login válido e inválido
- BcryptPasswordHasherTest — hash e conferência da senha
- AuthControllerTest, RestaurantControllerTest, MenuItemControllerTest — delegação dos controllers
- GlobalExceptionHandlerTest — ProblemDetail (400, 404, 409 e 500)
- ArchitectureTest — regra de dependência entre as camadas
- ManagementApplicationTests — carga do contexto

11. Considerações finais

A Fase 2 acrescenta os três cadastros pedidos sem descartar o módulo de usuários da Fase 1. O tipo deixa de ser uma herança de classes e passa a ser um registro associado ao usuário, que é o que o CRUD de tipo exige. Restaurante e item de cardápio referenciam, respectivamente, o dono e o restaurante.

A separação em domain, application e infrastructure deixa a regra de negócio fora do JPA e do HTTP. O ArchUnit registra essa regra no teste.

O ponto de atenção na execução é o banco criado na Fase 1: a coluna users_type permanece e o cadastro novo falha até o volume ser recriado.

12. Artefatos

- Código-fonte (Spring Boot / Java 21)
- docker-compose.yml, Dockerfile e .env.example
- Coleção Postman em docs/postman/RestaurantManagement.fase2.2.postman_collection.json
- Swagger em http://localhost:8080/swagger-ui.html e http://localhost:8080/v3/api-docs
- Testes em src/test/java
- Este relatório
- README do repositório, com o resumo de execução

Repositório: https://github.com/rodrfit/restaurant-management

13. Referências

- SPRING. Spring Boot Reference Documentation. https://docs.spring.io/spring-boot/
- IETF. RFC 7807 — Problem Details for HTTP APIs. https://datatracker.ietf.org/doc/html/rfc7807
- SPRINGDOC. springdoc-openapi. https://springdoc.org/
- DOCKER. Docker Compose overview. https://docs.docker.com/compose/
- MARTIN, Robert C. Clean Architecture: A Craftsman's Guide to Software Structure and Design. Prentice Hall, 2017.